

Proyecto 0: Mochila Dinámica vs. Mochila *Greedy*

Viernes 18 de Setiembre

I. DESCRIPCIÓN GENERAL

Se debe programar el algoritmo visto en clases para resolver el Problema 0/1 de la Mochila (*Knapsack*) usando programación Dinámica, junto con 2 versiones *greedy* del algoritmo. Los programas tendrán modo “ejemplo” donde generarán y resolverán un solo caso aleatorio que se resuelve con los 3 algoritmos, y un modo “experimento” donde generarán y resolverán muchísimos problemas de los que recolectarán datos estadísticos. En ambos casos, se produce automáticamente un documento latex con los resultados. Adicionalmente, cada grupo debe escribir un reporte respecto a sus conclusiones de las versiones *greedy* y dinámica del problema de la mochila. Toda la programación debe realizarse en C sobre Linux. No se pueden cambiar las especificaciones de este documento.

II. ANTECEDENTES

Deben leer del Winston (versión que se les envió) “*A Turnpike Theorem*” en las páginas 983 y 984. También deben estudiar el ejercicio 6 de la página 985 del mismo libro.

III. ALGORITMOS

Se deben considerar los siguientes algoritmos:

1. **Algoritmo de Programación Dinámica:** este es el algoritmo visto en clases para el caso 0/1.
2. **Algoritmo *greedy* básico:** cada vez se escoge el objeto más valioso que quepa en lo que sobre de capacidad de la mochila.
3. **Algoritmo *greedy* proporcional:** para cada objeto se calcula su rendimiento definido como el valor del objeto dividido entre la capacidad que consume. Entonces, cada vez se escoge el objeto de máximo rendimiento que quepa en lo que sobre de capacidad de la mochila.

IV. SALIDA

En todas las formas posibles de ejecución de este programa, siempre se debe generar un archivo “.tex” que será compilado con `pdflatex` o algún equivalente para que se produzca un archivo “pdf” que debe ser desplegado usando `evince` o algún equivalente. La invocación de todos estos comandos debe realizarse internamente desde su programa, de manera transparente para el usuario. Deben quedar disponibles tanto el pdf como el fuente de $\text{\LaTeX}$. Se espera que este documento tenga una presentación excelente, usando todas las capacidades de $\text{\LaTeX}$.

V. MODO EJEMPLO

Si al invocar el programa se pone en la línea de argumento la opción “-X” el programa resolverá un solo caso aleatorio con los 3 algoritmos mencionados. Este problema será creado por su programa, con las siguientes características:

- La mochila tendrá una capacidad de 15.
- Habrá 6 objetos. Se generarán aleatoriamente los c_i y v_i de cada objeto, con las restricciones de que $0 < c_i \leq 7$ y $0 < v_i \leq 20$.

Nótese que los 3 algoritmos resuelven el mismo caso. Todos deben reportar la solución encontrada con todo detalle junto con su tiempo de ejecución¹. Además el algoritmo de Programación Dinámica también debe presentar la tabla usada para sus cálculos (con entradas en verde y rojo). También se debe mostrar el problema inicial en forma matemática.

Tabla 1: Casos para Modo Experimento

Mochila	Cantidad de Objetos									
	10	20	30	40	50	60	70	80	90	100
100										
200										
300										
400										
500										
600										
700										
800										
900										
1000										

VI. MODO EXPERIMENTO

Si al invocar el programa se pone en la línea de argumento la opción “-E=n” el programa resolverá $100n$ casos diferentes con los 3 algoritmos mencionados. La capacidad de la mochila se varía desde 100 hasta 1000, y la cantidad de objetos se varía desde 10 hasta 100 objetos (ver **Tabla 1**).

Para cada uno de los n casos en la combinación de cierta capacidad de mochila y cantidad de objetos, se generarán aleatoriamente los c_i y v_i de cada objeto, con las restricciones de que todos los c_i sean mayores que 0, pero que ninguno sea mayor al 40% de la capacidad de la mochila, y que $0 < v_i \leq 100$.

Nótese que los 3 algoritmos resuelven los mismos $100n$ casos. Queremos recolectar los tiempos de ejecución promedio con la máxima precisión posible de las n ejecuciones en las 100 situaciones mostradas en la **Tabla 1**, para cada uno de los algoritmos. Esto es, habrá 3 tablas de tiempos.

¹Usar la máxima precisión posible. Por ejemplo, tiempos en segundos son inaceptables.

Por otro lado, sabemos que el algoritmo de Programación Dinámica encontrará la solución óptima cada vez y las versiones *greedy* ocasionalmente llegarán al mismo resultado. Necesitamos saber en que porcentaje de cada una de las 100 situaciones cada algoritmo *greedy* coincide en el resultado final del valor de la mochila con la solución óptima. Esto implica 2 tablas adicionales.

Todas las tablas mencionadas, junto con las explicaciones necesarias deben aparecer en el documento $\text{\LaTeX}$ generado por su programa.

VII. REPORTE ESCRITO

Después de muchas ejecuciones de su programa en modo “experimento” cada grupo habrá reunido evidencia respecto al comportamiento de los 3 algoritmos. Se debe escribir un reporte donde se analicen estos resultados y se lleguen a conclusiones interesantes, respecto a la idoneidad de cada algoritmo en diversas circunstancias. Este reporte se debe entregar en forma impresa en el instante de su revisión. Debe incluir tablas significativas generadas por su programa.

VIII. REQUISITOS INDISPENSABLES

La ausencia de uno solo de los siguientes requisitos vuelve al proyecto “no revisable” y recibe un 0 de calificación inmediata:

- La colaboración entre grupos se considera fraude académico.
- Utilizar código, archivos de datos, pruebas o documentación de semestres previos (*e.g.*, buscando en GitHub) o usar IA para completar cualquier parte de su trabajo no sólo es fraude académico, es una aceptación de incompetencia e inutilidad.
- Todo el código debe estar escrito en C (no C++).
- El proyecto debe compilar y ejecutar en Linux. Todo debe estar **integrado**, explicaciones del tipo “*todo está bien pero no pudimos pegarlo*”² provocan la cancelación automática de la revisión.
- La presentación debe ser de mucha calidad.
- Para el momento de la revisión, su programa no debe tener “prints” de depuración. Luce muy poco profesional.
- No debe dar “Segmentation Fault” bajo ninguna circunstancia.
- La única interacción válida con el programa es por medio de argumentos de línea de comando.
- La demostración debe hacerse en una máquina que levante Linux de manera real (puede ser dual), es decir no usar máquinas virtuales. No se puede levantar de un disco o memoria externa. No se pueden prestar máquinas entre grupos
- El día de la demostración se entrega el reporte escrito impreso.

²esto incluye los supuestos casos cuando alguien del grupo de trabajo no hizo su parte – el profesor no está interesado en sus problemas de organización.

- Se debe generar $\text{\LaTeX}$ y el reporte final debe estar escrito en $\text{\LaTeX}$.

IX. FECHA DE ENTREGA

Revisiones a las 11:30am el **Viernes 18 de Septiembre** en la oficina del pfsor. Mande además un .tgz con todo lo necesario (reporte, fuentes, makefile, readme, etc.) a torresrojas.cursos.04@gmail.com. Ponga como subject: [IO]– Proyecto 0 – Fulano – Mengano, donde Fulano y Mengano son los 2 miembros del grupo.